



Translator WebApp mit Terraform und Ansible

Cloud Computing 2 – Abschlussprojekt

Bachelor of Science

des Studiengangs IT-Automotive

an der Dualen Hochschule Baden-Württemberg Stuttgart

von

Tim Baßmann

20.04.2025

Bearbeitungszeitraum
Matrikelnummer, Kurs
Dozent

24.02.2025 - 20.04.2025
4274403, TINF22ITA
Nils Riekers

Inhaltsverzeichnis

Abkürzungsverzeichnis	II
Abbildungsverzeichnis	III
1 Einleitung	1
1.1 Problemstellung	1
1.2 Aufgabenstellung	1
2 Stand der Technik	2
2.1 Entwicklung der Webanwendung	2
2.1.1 Flask	2
2.1.2 uWSGI	2
2.1.3 MongoDB Atlas	3
2.1.4 Azure Translator	3
2.2 Deployment der Webanwendung	3
2.2.1 Docker	3
2.2.2 Terraform	4
2.2.3 Ansible	4
3 Implementierung	5
3.1 Architektur	5
3.2 Webanwendung	5
3.2.1 Frontend	5
3.2.2 Backend	6
3.3 Docker	6
3.3.1 Dockerfile	6
3.3.2 Docker Compose	7
3.4 Deployment	7
3.4.1 Terraform	7
3.4.2 Ansible	8
4 Fazit und Ausblick	9
4.1 Fazit	9
4.2 Ausblick	9
Quellen	A
Anhang	B

Abkürzungsverzeichnis

AI	Artificial Intelligence
API	Application Programming Interface
CI	Continuous Integration
CD	Continuous Deployment
HTML	Hypertext Markup Language
IaaS	Infrastructure as a Service
IaC	Infrastructure as Code
IP	Internet Protocol
IPA	Internet Protocol Address
IT	Information Technology
PaaS	Platform as a Service
SaaS	Software as a Service
JSON	JavaScript Object Notation
NSG	Network Security Group
PC	Personal Computer
SSH	Secure Shell
VM	Virtual Machine
WSGI	Web Server Gateway Interface
YAML	Yet Another Markup Language

Abbildungsverzeichnis

3.1	Architektur und Deployment/Provision der einzelnen Bestandteile	5
A1	Die Übersetzer-Webanwendung	B

1 Einleitung

1.1 Problemstellung

Die Cloud-Technologie hat in den letzten Jahren erheblich an Bedeutung gewonnen. Unternehmen und Organisationen nutzen zunehmend Cloud-Dienste, um ihre Information Technology (IT)-Infrastruktur zu optimieren. Dabei gibt es verschiedene Modelle, die unterschiedliche Anforderungen und Bedürfnisse abdecken. Die drei Hauptmodelle sind Infrastructure as a Service (IaaS), Platform as a Service (PaaS) und Software as a Service (SaaS). Diese Modelle bieten unterschiedliche Ebenen der Abstraktion und Kontrolle über die zugrunde liegende Infrastruktur.

Da diese verschiedenen Systeme immer komplexer werden, ist es umso wichtiger, dass Unternehmen ihre IT-Infrastruktur wiederholbar, effizient und kostengünstig aufbauen können. Hierfür eignet sich das Verwenden von Infrastructure as Code (IaC)-Tools, um die Bereitstellung der Infrastruktur automatisiert wird.

1.2 Aufgabenstellung

Die Aufgabe dieser Arbeit besteht darin, die drei Aspekte IaaS, PaaS und SaaS anhand eines Praxisbeispiels verwenden zu können.

Hierfür soll für den SaaS-Anteil eine Webanwendung entwickelt werden, die es Nutzern ermöglicht, Texte in verschiedene Sprachen zu übersetzen. Diese Webanwendung soll dann auf einer Microsoft Azure VM bereitgestellt werden, um den Aspekt von IaaS zu erfüllen. Um letztendlich ebenfalls noch den PaaS-Aspekt mit integrieren zu können, soll mit einer Datenbank kommuniziert werden, um bereits getätigte Anfragen zwischenspeichern zu können. Das Deployment von den Services soll mithilfe von Terraform und Ansible automatisiert werden.

2 Stand der Technik

2.1 Entwicklung der Webanwendung

Die Entwicklung der Webanwendung erfolgt in Python, wobei das Flask-Framework verwendet wird. Die Anwendung wird auf einem uWeb Server Gateway Interface (WSGI)-Server bereitgestellt, der die Kommunikation zwischen dem Webserver und der Anwendung ermöglicht. Diese Kombination wird in einem Docker-Container betrieben, um eine einfache Bereitstellung und Skalierung zu gewährleisten.

2.1.1 Flask

Flask ist ein leichtgewichtiges Framework für Python, das eine einfache und flexible Möglichkeit bietet, Webanwendungen zu erstellen. Es ist bekannt für seine Einfachheit und Modularität, was es Entwicklern ermöglicht, schnell Prototypen zu erstellen. Andererseits ist es ebenfalls möglich, im Laufe der Entwicklung die Anwendungen zu skalieren. Flask folgt dem WSGI-Standard und ist in der Lage, mit verschiedenen Webservern zu kommunizieren. Zudem werden Jinja Templates unterstützt, die es ermöglichen, HTML-Seiten dynamisch zu generieren. [1]

2.1.2 uWSGI

uWSGI ist ein Webserver, der speziell für die Ausführung von Python-Anwendungen entwickelt wurde. Es ist eine Implementierung des WSGI-Standards und ermöglicht es Entwicklern, ihre Anwendungen effizient bereitzustellen. uWSGI bietet eine Vielzahl von Funktionen, darunter Lastverteilung, Caching und Unterstützung für verschiedene Protokolle. Es ist bekannt für seine hohe Leistung und Skalierbarkeit und wird häufig in Produktionsumgebungen eingesetzt. [2]

2.1.3 MongoDB Atlas

MongoDB Atlas ist ein vollständig verwalteter PaaS-Cloud-Datenbankdienst, der von MongoDB bereitgestellt wird. Es ermöglicht Entwicklern, MongoDB-Datenbanken in der Cloud zu erstellen, zu verwalten und zu skalieren, ohne sich um die zugrunde liegende Infrastruktur kümmern zu müssen. Es unterstützt verschiedene Cloud-Anbieter wie AWS, Microsoft Azure und Google Cloud, was es zu einer flexiblen Lösung für moderne Anwendungen macht. MongoDB Atlas erleichtert die Verwaltung von Datenbanken erheblich, wodurch sich Entwickler auf ihre Anwendungen konzentrieren können. [3]

2.1.4 Azure Translator

Azure Translator ist ein KI-gestützter Übersetzungsdienst, der von Microsoft bereitgestellt wird. Er ermöglicht die Übersetzung von Texten in über 100 Sprachen und Dialekte. Der Dienst bietet Funktionen wie Echtzeitübersetzungen, benutzerdefinierte Übersetzungsmodelle und die Integration in Anwendungen über REST-APIs. Zudem unterstützt er die Übersetzung von Dokumenten und bietet eine hohe Genauigkeit durch den Einsatz modernster KI-Technologien. [4]

2.2 Deployment der Webanwendung

Die Webanwendung wird in einer IaaS-Umgebung bereitgestellt, die auf Microsoft Azure basiert. Diese Umgebung ermöglicht es, die Anwendung in einer skalierbaren und flexiblen Infrastruktur zu betreiben. Hierfür wird Terraform in Kombination mit Ansible verwendet. Die Bereitstellung erfolgt mithilfe eines Docker-Containers, der die Anwendung enthält. Dies erleichtert die Verwaltung und Skalierung der Anwendung in der Cloud.

2.2.1 Docker

Docker ist eine Plattform zur Automatisierung der Bereitstellung von Anwendungen in Containern. Container sind leichtgewichtige, tragbare und isolierte Umgebungen, die es Entwicklern ermöglichen, Anwendungen und ihre Abhängigkeiten zusammenzufassen. Docker bietet eine einfache Möglichkeit, Anwendungen zu erstellen, bereitzustellen und zu skalieren, indem es eine konsistente Umgebung über verschiedene Systeme hinweg bereitstellt. Dies erleichtert die Zusammenarbeit zwischen Entwicklern und Betriebsteams

und reduziert Probleme, die durch unterschiedliche Umgebungen verursacht werden können. Hierfür stellt Docker die sogenannte Docker-Engine zur Verfügung, welche zwischen dem Host-Betriebssystem und den Containern vermittelt. [5]

Docker Compose ist ein Werkzeug, das es ermöglicht, Multi-Container-Anwendungen zu definieren und auszuführen. Mit einer YAML-Datei können die Dienste einer Anwendung konfiguriert und mit einem einzigen Befehl gestartet werden. Es vereinfacht die Verwaltung von Anwendungen, die aus mehreren Containern bestehen, indem es eine deklarative Konfiguration und eine einfache Orchestrierung bietet. [6]

Mithilfe von Docker Multi-Stage Builds, können Container-Images in mehreren Schritten erstellt werden. Dies ermöglicht es, verschiedene Phasen des Build-Prozesses zu optimieren. Durch die Verwendung dieser Multi-Stage Builds können Entwickler sicherstellen, dass nur die benötigten Dateien und Abhängigkeiten im endgültigen Image enthalten sind. Abhängigkeiten, die nur zum Bauen der Bibliotheken oder ähnliches notwendig sind, können beispielsweise nur in der Build-Phase verwendet werden. Das erhöht letztendlich die Sicherheit, Effizienz und Größe des Container-Images. [7]

2.2.2 Terraform

Terraform ist ein Open-Source IaC-Tool, das es ermöglicht, Infrastruktur sicher und effizient bereitzustellen, zu ändern und zu verwalten. Es verwendet eine deklarative Konfigurationssprache, um die gewünschte Infrastruktur zu definieren, und erstellt einen Ausführungsplan, der beschreibt, wie Terraform die Infrastruktur erreicht. Terraform unterstützt eine Vielzahl von Cloud-Anbietern und anderen Diensten, was es zu einem vielseitigen Werkzeug für die Verwaltung von Infrastruktur in hybriden und Multi-Cloud-Umgebungen macht. [8]

2.2.3 Ansible

Ansible ist ein Open-Source-Automatisierungswerkzeug, das für Konfigurationsmanagement, Anwendungsbereitstellung und Orchestrierung verwendet wird. Es ermöglicht die Automatisierung von IT-Aufgaben, indem es eine deklarative Sprache verwendet, um gewünschte Zustände von Systemen zu definieren. Diese werden in einer Yet Another Markup Language (YAML) Datei namens „Playbook“ gespeichert. Für Ansible muss keine zusätzliche Software auf den Zielsystemen installiert werden. Stattdessen verwendet es Secure Shell (SSH), um mit den Systemen zu kommunizieren. [9]

3 Implementierung

3.1 Architektur

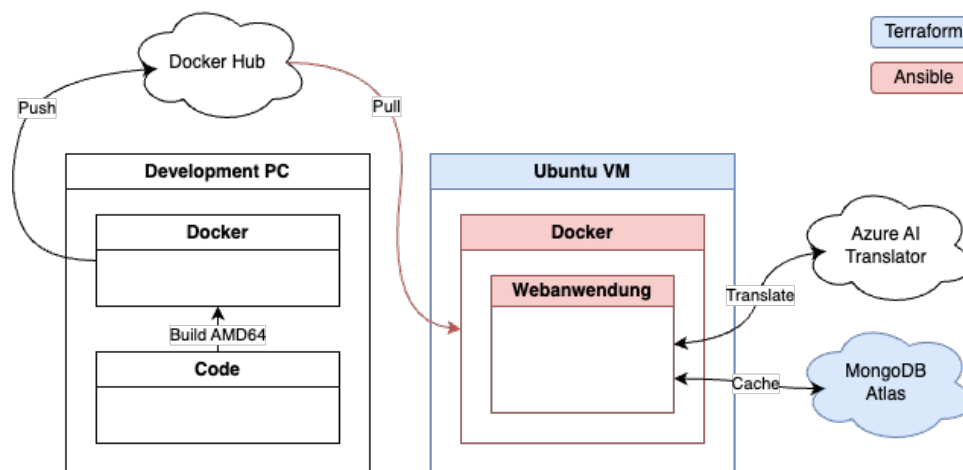


Abbildung 3.1: Architektur und Deployment/Provision der einzelnen Bestandteile [10]

Die Architektur des Projekts ist in Abbildung 3.1 dargestellt. In dieser ist ein Development Personal Computer (PC) aufgeführt, von welchem aus der Source Code entwickelt, gebaut und die Anwendung provisioniert wird. Der Code wird mithilfe von Docker containerisiert, für eine AMD64-Architektur – wie diese der Ubuntu Virtual Machine (VM) – gebaut und in den Docker Hub gepusht. Von dort wird dann in der erstellten VM die Webanwendung heruntergeladen und gestartet. Zudem kommuniziert die Anwendung zur Laufzeit mit einer MongoDB Datenbank und dem Microsoft Azure Artificial Intelligence (AI) Translator.

3.2 Webanwendung

3.2.1 Frontend

Eine Abbildung der fertigen Webanwendung ist im Anhang unter A1 zu finden. Diese zeigt oben eine Navigationsleiste, in welcher auf der rechten Seite die Sprache der Webseite und der Stil – hell, dunkel oder automatische Wahl – ausgewählt werden kann. Darunter befindet sich ein Feld zum Eingeben des zu übersetzenden Texts und ein Dropdown-Menü

zum Auswählen der Originalsprache. Im Kasten darunter wird – nach klicken auf den Knopf am unteren Rand – der übersetzte Text angezeigt.

Um diese Seite zu realisieren, werden Jinja Templates verwendet, welche beispielsweise die Namen der Sprachen in den Dropdown-Menüs einfügen. Zudem wird jeder String der Seite in einer Konstanten-Datei gespeichert, um diese einfach in eine auf der Webseite eingestellte Sprache übersetzen lassen zu können. Diese übersetzten Strings werden wiederum dynamisch im Jinja Template eingefügt. Das Aussehen der Webseite – bzw. der Hypertext Markup Language (HTML)-Elemente – wird mithilfe von Bootstrap angepasst.

3.2.2 Backend

Die Webseite verfügt generell über zwei Routen. Die erste Route ist die Startseite, auf welcher der Benutzer den Text eingeben kann, der übersetzt werden soll. Diese Route wird mit einem GET-Request aufgerufen und gibt die Webseite zurück. Die zweite Route ist die Übersetzungsrouten, welche mit einem POST-Request aufgerufen wird.

Diese Application Programming Interface (API)-Schnittstelle bekommt – beim Drücken des Übersetzen-Knopfes – den eingegebenen Text und die zugehörigen Sprachen vom Client und gibt den übersetzten Text zurück. Hierfür wird zuerst überprüft, ob der Text sich bereits in der Datenbank befindet. Falls dies der Fall ist, wird der Text von dort zurückgegeben. Andernfalls wird der Text an den Microsoft Azure AI Translator geschickt und dessen Antwort in der Datenbank gespeichert. Um möglichst effizient auf die Datenbank zugreifen zu können, besitzt jede Sprache in der MongoDB eine eigene Collection. Dadurch muss man bei der Abfrage nicht alle bereits gespeicherten Einträge durchgehen, sondern lediglich die der Ausgangssprache.

3.3 Docker

3.3.1 Dockerfile

Das Dockerfile wird als Multi-Stage-Build erstellt. Wie in Unterabschnitt 2.2.1 bereits erklärt, kann man damit Container-Images in mehreren Schritten erstellen, um diese so performant wie möglich zu halten. Zudem wird als Basis-Image die Linux Alpine Distribution gewählt, da diese nur die nötigsten Pakete beinhaltet. Das erhöht auf der einen Seite die Sicherheit, da ungenutzte Pakete, die mögliche Sicherheitslücken enthalten, nicht installiert sind. Auf der anderen Seite wird das Image dadurch zudem weiter verkleinert. Innerhalb des Dockerfiles werden außerdem zusammengehörende Schritte zusammengefasst,

um die Anzahl der Layer zu minimieren und damit ebenfalls das Image weiter zu verkleinern. Um ebenfalls während der Entwicklung Zeit zu sparen, werden die Ebenen so aufgebaut, dass sich nur die letzten Schritte ändern, wenn sich der Source Code ändert. Das bedeutet, dass die ersten Schritte, wie das Installieren der Abhängigkeiten, nicht erneut ausgeführt werden müssen, wenn sich nur der Source Code ändert. Dies liegt daran, dass Docker während dem Bauen die einzelnen Ebenen cachen kann.

3.3.2 Docker Compose

Um das Docker-Image letztendlich deployen zu können, wird das für die Architektur AMD64 gebaute Image in den Docker Hub gepusht. Ausgeführt wird dieses auf der VM dann mithilfe eines Docker Compose Files. Für die Entwicklung wurde ein eigenes Compose File erstellt, welches es ermöglicht, die Dateien im Container mit denen auf dem Host zu synchronisieren. Das bedeutet, dass Docker automatisiert die Dateien neu lädt oder sogar den kompletten Container neu baut, wenn sich eine Datei auf dem Host ändert.

Im Compose File für die Produktion, wird das Image vom Docker Hub geladen, der Port 5000 innerhalb des Containers wird auf Port 80 des Hosts gesetzt und ein `.env`-File wird geladen, um Umgebungsvariablen – darunter Sicherheitsschlüssel – zu setzen.

3.4 Deployment

3.4.1 Terraform

Der IaC-Service „Terraform“ wird – wie in Abbildung 3.1 blau gekennzeichnet – verwendet, um die VM in Microsoft Azure und die MongoDB in Atlas zu provisionieren. Um die beiden Dienste zu trennen, wird jeweils ein eigenes Modul in der `main.tf`-Datei angelegt und Variablen an diese übergeben.

Bevor diese Module allerdings ausgeführt werden können, müssen zuerst die im `.env`-File gespeicherten Umgebungsvariablen geladen werden. Diese beinhalten verschiedenste Schlüssel wie unter anderem den API-Key für den Microsoft Azure Translator. Um diese Umgebungsvariablen direkt in den Run der `main.tf`-Datei einzubinden, wird ein Shell-Skript verwendet, welches die Variablen aus dem `.env`-File ausliest und diese im JavaScript Object Notation (JSON)-Format an die aufrufende `main.tf`-Datei zurückgibt.

Daraufhin wird für die spätere Verwendung von Ansible ein Playbook mithilfe eines Templates erstellt. In diesem Template werden dabei die Internet Protocol Address (IPA) der Azure VM und der passende Benutzername eingetragen. Zuletzt werden dann die von

den Modulen erzeugten Ausgaben – teils als sensitiv markiert, sodass diese nicht direkt einsehbar sind – ausgegeben, um diese später verwenden zu können.

Im Modul für die Azure VM wird der VM eine statische, öffentliche IPA zugewiesen, um diese später im Ansible Playbook und zur Verbindung zum Translator verwenden zu können. Um die Verbindungen auf Port 80 für die Webanwendung und Port 22 für SSH zu ermöglichen, werden zwei weitere Regeln in die Network Security Group (NSG) geschrieben. Bei Port 22 sind allerdings nur die IPAs erlaubt, die zuvor in das `.env`-File geschrieben wurden. Diese IPAs sollten mindestens die öffentliche Adresse des Development PCs enthalten.

Das Modul für die MongoDB in Atlas erstellt ein neues Projekt, in diesem eine Datenbank und einen Benutzer, welcher die nötigen Rechte besitzt, um Werte in der Datenbank ändern zu können. Zudem wird die IPA des Azure VM in die Whitelist der MongoDB geschrieben, um dieser zu erlauben, auf die Datenbank zugreifen zu dürfen.

3.4.2 Ansible

Ansible wird verwendet, um die VM zu provisionieren – in Abbildung 3.1 rot gekennzeichnet. Hierbei wird ein Playbook verwendet, welches die IPA und den Benutzernamen der VM aus dem Terraform-Run enthält.

Um das in der Docker Compose-Datei enthaltene `.env`-File verwenden zu können, muss dieses zuerst erstellt werden. Hierfür wird ein eigenes Shell-Skript verwendet, welches die Umgebungsvariablen aus dem Terraform-Run in das `.env`-File schreibt. Dieses Skript wiederum wird von einem weiteren Shell-Skript aufgerufen, welches sowohl die Variablen aus dem Terraform-Run ausliest und dem ersten Skript übergibt, als auch das Ansible Playbook ausführt.

Im Ansible Playbook werden zuerst die nötigen Pakete installiert. Daraufhin wird der Benutzer der VM zu der Docker-Gruppe hinzugefügt und überprüft, ob der Docker Demon läuft. Daraufhin wird ein Ordner für die Anwendung erstellt und die Docker Compose-Datei für den Produktionsbetrieb, zusammen mit dem zuvor durch das Shell-Skript erzeugte `.env`-File, in diesen Ordner kopiert. Zuletzt wird der Container gestartet und die IPA der VM in der Konsole zurückgegeben.

4 Fazit und Ausblick

4.1 Fazit

In dieser Arbeit wurde eine Webanwendung als SaaS zur Übersetzung von Texten in verschiedene Sprachen entwickelt. Diese Anwendung nutzt den Microsoft Azure AI Translator und speichert die übersetzten Texte in einer MongoDB Datenbank, um den PaaS-Aspekt abzudecken. Um die Anwendung skalierbar machen zu können, wurde Docker verwendet. Das Deployment der Anwendung erfolgt mithilfe von Terraform und Ansible auf einer Microsoft Azure VM als IaaS.

4.2 Ausblick

Das Projekt bietet an sich eine gute Basis für eine Webanwendung, die mithilfe von Terraform und Ansible provisioniert werden kann. Um diese Anwendung dennoch weiter zu verbessern und produktionsreif zu machen, könnten folgende Punkte umgesetzt werden:

- **Load Balancer:** Um die Anwendung skalierbar zu machen, könnte ein Load Balancer zwischen der VM und dem Client eingesetzt werden. Dadurch könnte die Anwendung auf mehrere VMs verteilt werden, um die Last besser zu verteilen.
- **Reverse-Proxy:** Um die Anwendung sicherer zu machen, könnte ein Reverse-Proxy eingesetzt werden. Dieser könnte die Anfragen an die VM weiterleiten und dabei SSL-Zertifikate verwenden, um die Kommunikation zu verschlüsseln.
- **Monitoring-Tools:** Um die Anwendung zu überwachen, könnten verschiedene Monitoring-Tools eingesetzt werden. Diese könnten die Leistung der Anwendung überwachen und bei Problemen Alarm schlagen.
- **Continuous Integration (CI)/Continuous Deployment (CD)-Pipelines:** Um die Anwendung kontinuierlich zu integrieren und zu deployen, könnten verschiedene CI/CD-Tools eingesetzt werden. Diese könnten den Code automatisch testen und bei Änderungen automatisch deployen.

Quellen

- [1] Pallets, *Flask*, 2010. besucht am 6. Apr. 2025. Adresse: <https://flask.palletsprojects.com/en/stable/>.
- [2] uWSGI, *The uWSGI project*, 2016. besucht am 6. Apr. 2025. Adresse: <https://uwsgi-docs.readthedocs.io/en/latest/>.
- [3] MongoDB, Inc., *Database. Bereitstellen einer Multi-Cloud-Datenbank.* de, 2025. besucht am 7. Apr. 2025. Adresse: <https://www.mongodb.com/de-de/products/platform/atlas-database>.
- [4] Microsoft, *Azure AI Translator*, en-US, 2025. besucht am 7. Apr. 2025. Adresse: <https://azure.microsoft.com/en-us/products/ai-services/ai-translator>.
- [5] Amazon Web Services, Inc., *Was ist Docker?* de-DE, 2024. besucht am 7. Apr. 2025. Adresse: <https://aws.amazon.com/de/docker/>.
- [6] Docker, Inc., *Docker Compose*, en, Nov. 2024. besucht am 8. Apr. 2025. Adresse: <https://docs.docker.com/compose/>.
- [7] Docker, Inc., *Multi-stage builds*, en, 2025. besucht am 7. Apr. 2025. Adresse: <https://docs.docker.com/build/building/multi-stage/>.
- [8] HashiCorp, *What is Terraform?* en. besucht am 7. Apr. 2025. Adresse: <https://developer.hashicorp.com/terraform/intro>.
- [9] Ansible project contributors, *Introduction to Ansible*, en, März 2025. besucht am 7. Apr. 2025. Adresse: https://docs.ansible.com/ansible/latest/getting_started/introduction.html.
- [10] T. Baßmann, *Eigene Darstellung*, Apr. 2025.

Anhang

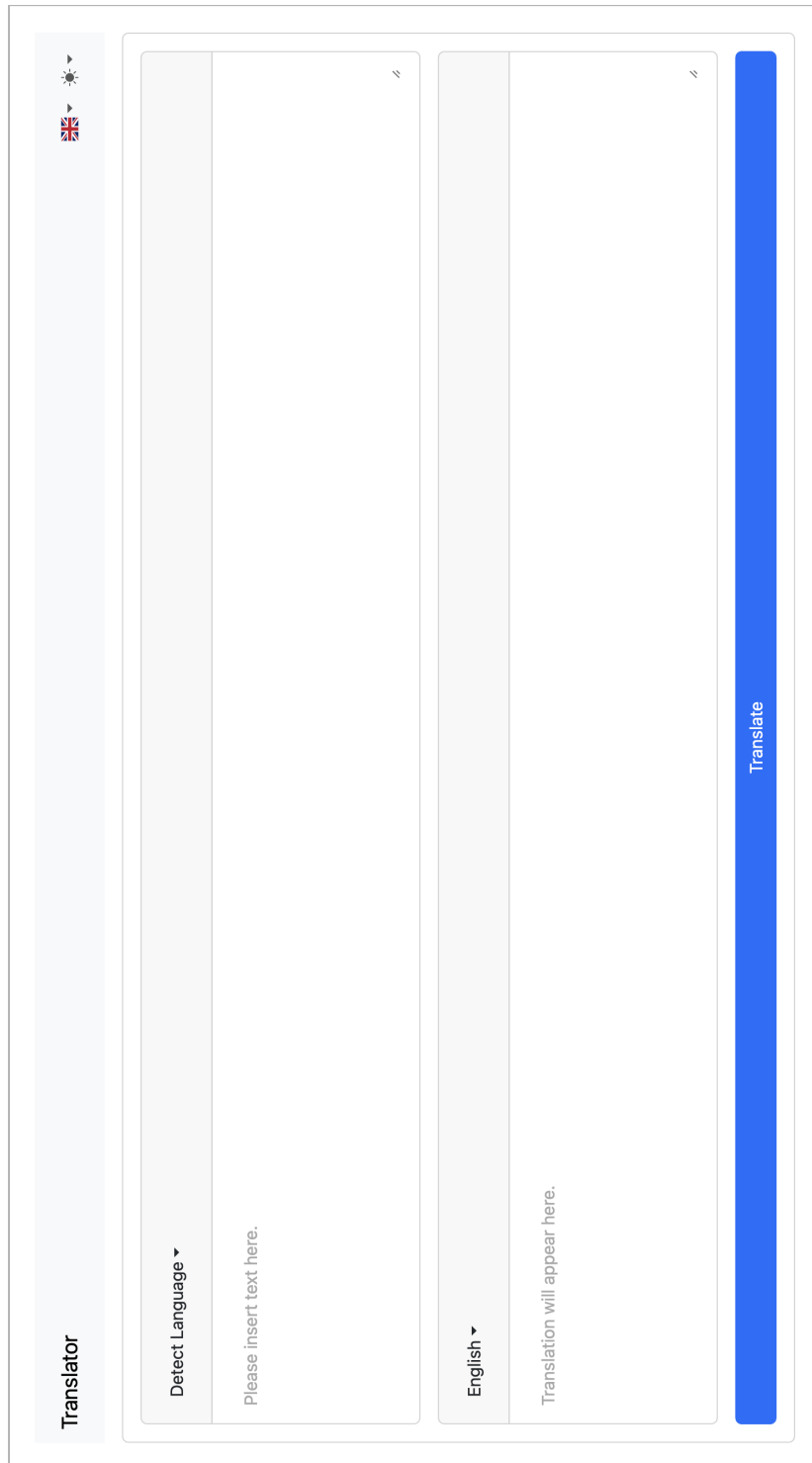


Abbildung A1: Die Übersetzer-Webanwendung [10]